Clustering:

Clustering is the part of unsupervised learning which is based on detecting the groups and with no labeled data .

What is K-Means Clustering

K-Means is an algorithm in
Machine Learning
that divides data into **K clusters** based on similarity.

"K" = number of groups you want

K-Means says:

1. Pick K centers
2. Assign each point to nearest center
3. Move centers
4. Repeat

Step-by-Step Working

Step 1: Choose K

Example:
K = 2 (we want 2 clusters)

Step 2: Initialize Centroids

Pick random points as centers

Example:

- C1 = (2, 2)
- C2 = (8, 8)

Step 3: Assign Points to Nearest Centroid

Distance formula:

d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}

What is K-Means Algorithm?

K-Means is an algorithm in
Machine Learning
used to **partition data into K clusters** such that:

Each data point belongs to the cluster with the **nearest mean (centroid)**

How K-Means Works

1. **Initialization**
   Choose **K initial centroids**, either randomly or using a heuristic (like smarter initialization methods).

2. **Assignment Step**
   Assign each data point to the **nearest centroid**, forming K clusters.
   Distance is usually measured using:

   d = sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}

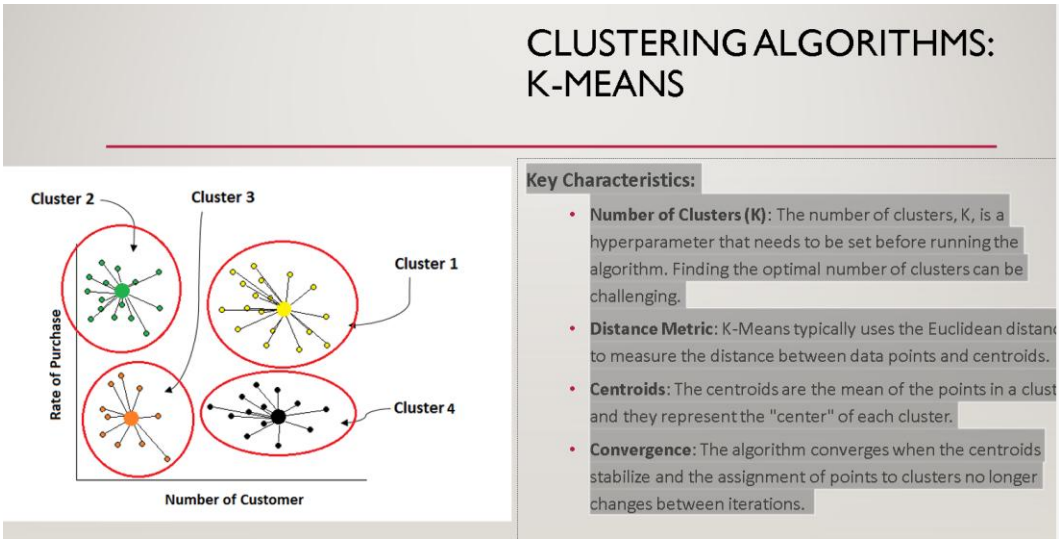
3. **Update Step**
   Recalculate each centroid as the **mean of all points** in that cluster.

4. **Iteration (Repeat Process)**
   Repeat:
   - Assignment step
   - Update step

   Until **convergence**, meaning:

   - Centroids stop changing significantly, OR
   - Cluster assignments remain the same

“K-Means iteratively minimizes the distance between data points and their respective cluster centroids.”



Number of Clusters (K)

- K is a **hyperparameter** that must be defined before running the algorithm.
- Choosing the optimal K can be challenging.
- Common method used:
  - **Elbow Method**

Again let see

What is the Elbow Method?

The Elbow Method is used in
Machine Learning
to find the **best value of K** in K-Means.

Advantages of K-Means:

- **Efficiency**: It is computationally efficient for large datasets, especially when the number of features and clusters is not too large.

- **Ease of Implementation**: The algorithm is straightforward to implement and understand.
- **Scalability**: It can be scaled to large datasets with some modifications.

Disadvantages:

- **Sensitivity to Initial Conditions**: The initial placement of centroids can affect the final clusters. Therefore, it is common to run K-Means multiple times with different initializations and choose the best result.
- **Spherical Cluster Shape Assumption**: K-Means assumes that clusters are spherical and evenly sized, which is not always the case.

We can now connect this to:

1. **Underfitting vs Overfitting**
2. **Why K-Means fails in real data**
3. **DBSCAN vs K-Means (better understanding)**
4. **Silhouette Score (advanced evaluation)**

1. Underfitting vs Overfitting (Connected to K-Means)

Underfitting

- Model is too simple
- Misses patterns

In K-Means:

- K is too small (e.g., K = 1 or 2)
- Everything grouped badly

Overfitting

- Model is too complex
- Fits noise

In K-Means:

- K is too large (e.g., K = number of data point

Definition (from original paper by Martin Ester et al., 1996):
DBSCAN is an unsupervised clustering algorithm that defines clusters as **dense regions of points** separated by regions of lower Noise. A cluster is formed if a point has at least **MinPts** neighbors within a radius **ε (epsilon)**.

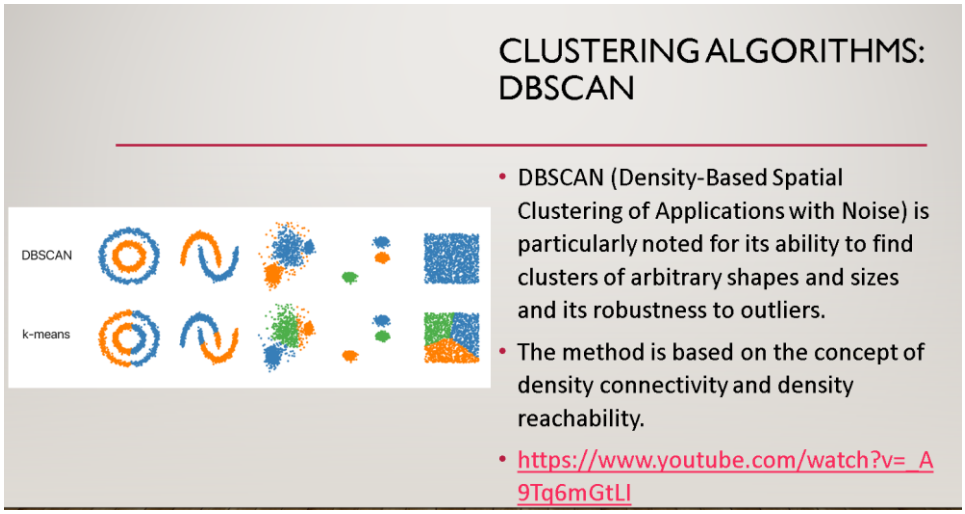
DBSCAN is all about density and noise

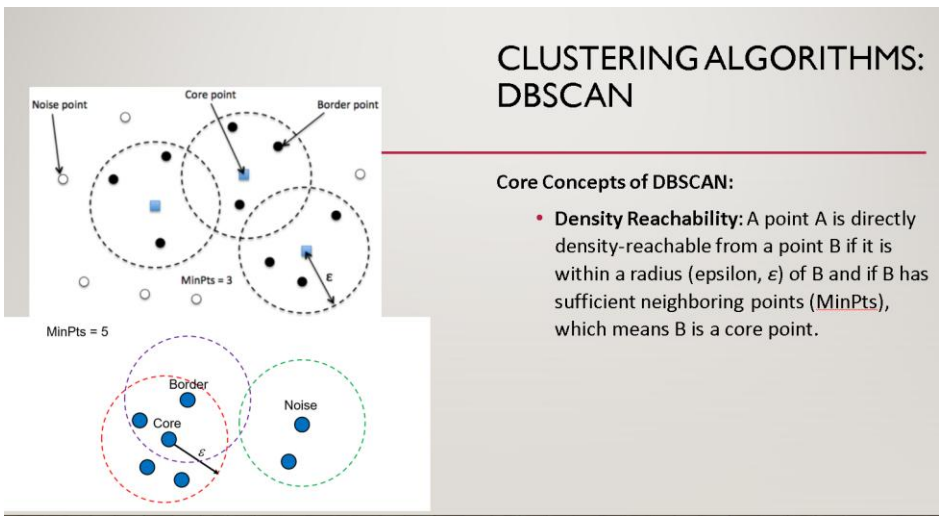
- DBSCAN **does not rely on centroids** like K-Means
- It **defines clusters based on density**
- Everything else that doesn't meet density requirements → considered **noise**

How Noise Works in DBSCAN(watch me)

Two key parameters:

1. **ε (epsilon)** – radius of neighborhood
    - Larger ε → more points considered neighbors → fewer points classified as noise → **bigger clusters**
    - Smaller ε → fewer neighbors → more points are noise → **smaller clusters or isolated noise points**
2. **MinPts** – minimum points required to form a cluster
    - Larger MinPts → stricter → more noise points
    - Smaller MinPts → easier to form clusters → less noise





"DBSCAN forms clusters based on density reachability: core points define dense regions,

border points attach to clusters, and points that are neither core nor reachable are classified as noise."

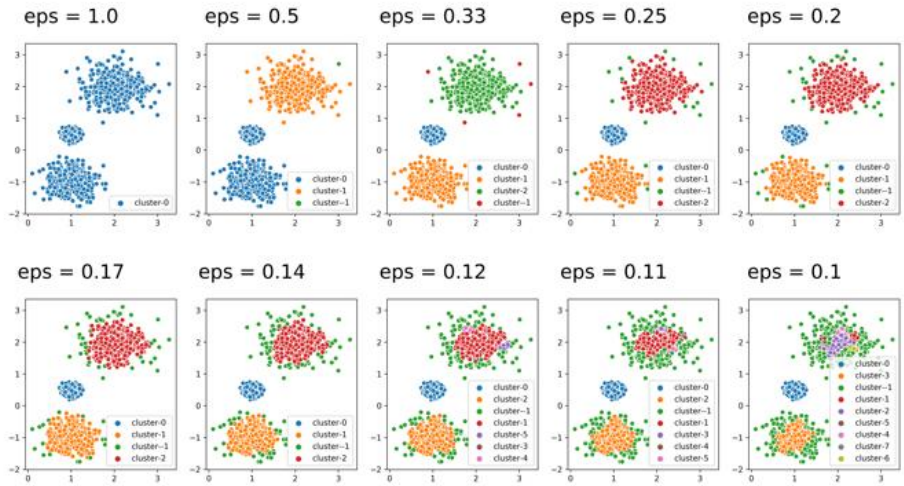
Clustering Algorithms:
DBSCAN

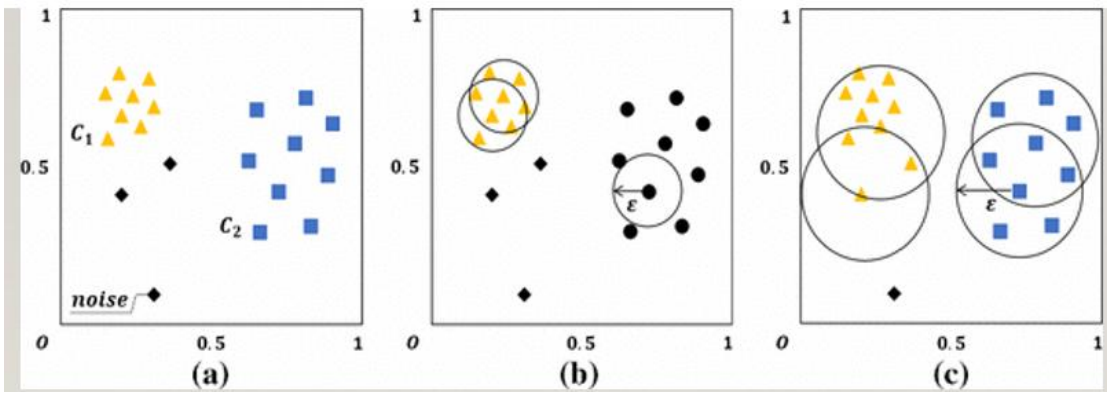
Steps

- **Choose Parameters**: Set the two parameters ε (epsilon) and MinPts.
- **Find Core Points**: For each point in the dataset, count how many points fall within the ε-neighborhood. If a point has at least MinPts within its ε-neighborhood, it is marked as a core point.
- **Expand Clusters**: For each core point, if it is not already assigned to a cluster, create a new cluster, then recursively find and merge all its density-reachable points to form a single cluster.
- **Identify Noise**: Any point that is not a core point and is not density-reachable from any core point is considered noise and not included in any cluster.

Advantages of DBSCAN:

- **No Need to Specify the Number of Clusters**: Unlike K-means, DBSCAN does not require you to specify the number of clusters beforehand.
- **Ability to Find Arbitrarily Shaped Clusters**: DBSCAN can find clusters of arbitrary shapes, whereas K-means assumes spherical clusters.
- **Robustness to Outliers**: Points classified as noise by DBSCAN can be considered outliers, making the algorithm robust to outliers.
- **Only Two Parameters**: DBSCAN only requires two parameters, ε and MinPts, which are typically easier to estimate than the number of clusters.



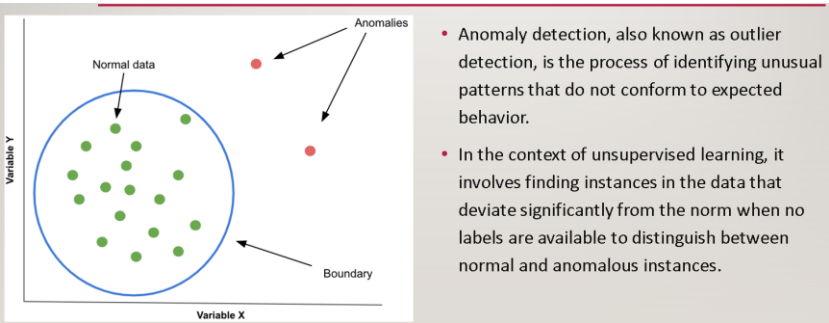


Disadvantages:

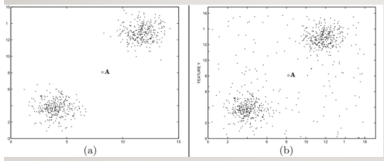
- **Parameter Sensitivity**: Choosing an appropriate value for ε can be challenging and heavily influences the resulting clusters. Similarly, MinPts needs to be chosen carefully, especially in datasets with varying densities.
- **Difficulty with Varying Densities**: DBSCAN can struggle when the clusters have widely different densities. Such scenarios can lead to a merging of clusters that should be separate, or to dense regions being split into multiple clusters.
- **Computational Complexity**: While more efficient than some other clustering algorithms, DBSCAN can still be computationally intensive, especially on large datasets with many dimensions.

Anomaly Detection

“Anomaly detection identifies data points that deviate significantly from normal behavior. In unsupervised learning, it relies on patterns and densities in the data without needing labels.”



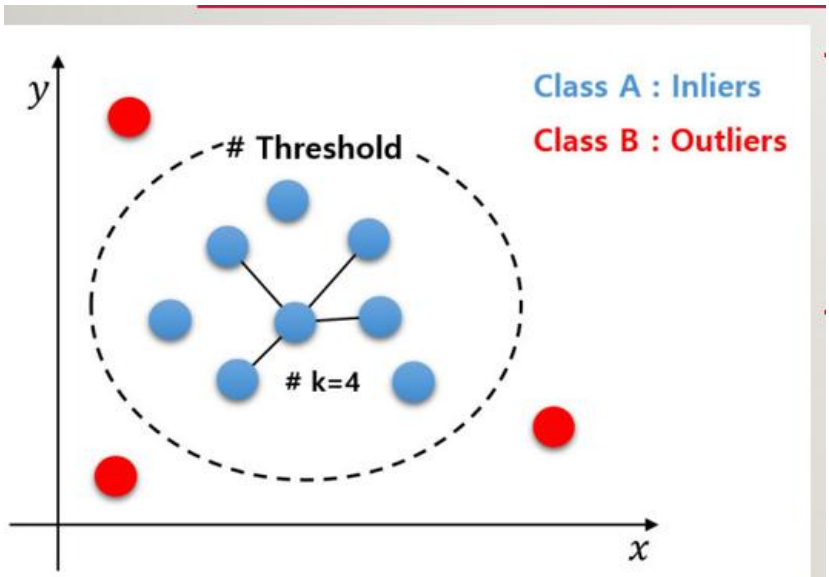
- Anomaly detection, also known as outlier detection, is the process of identifying unusual patterns that do not conform to expected behavior.
- In the context of unsupervised learning, it involves finding instances in the data that deviate significantly from the norm when no labels are available to distinguish between normal and anomalous instances.



Key Concepts in Anomaly Detection:

- **Anomalies (Outliers)**: Data points that are significantly different from the majority of the data. They can be due to measurement error, system faults, or simply natural deviations in populations.
- **Novelty Detection**: Sometimes a distinction is made between outlier detection and novelty detection. The latter refers to identifying new or unknown patterns that were not available during the training phase.
- **Noise**: Anomaly detection must distinguish between genuine anomalies and random noise or variability in the data.

Anomaly Detection Methods

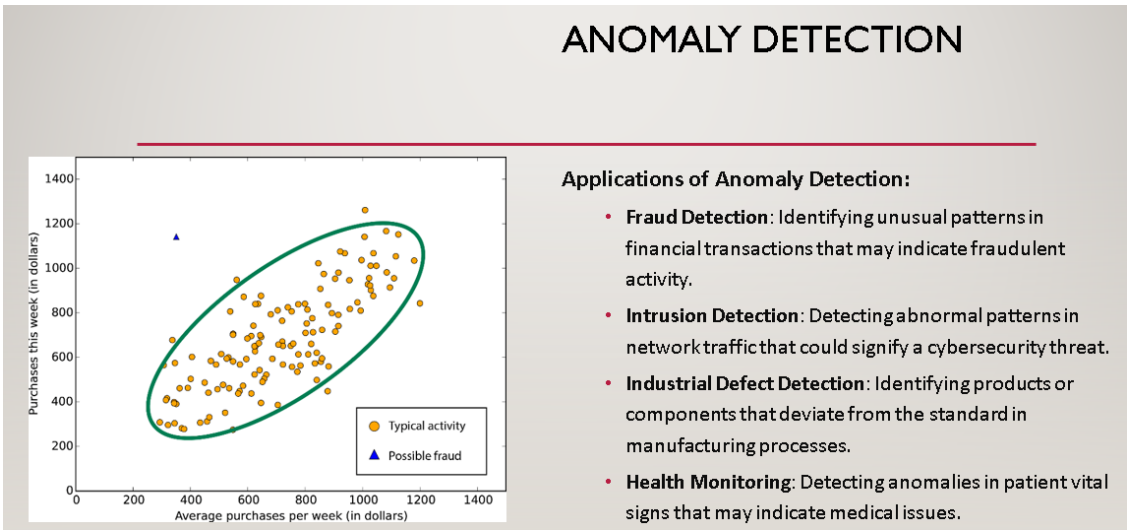


- **Statistical Approaches**:
  - These methods model the normal behavior using statistical models and then find instances with low probability under the model.
  - Commonly used statistical methods include z-scores and chi-squared,
- **Machine Learning Approaches**:
  - **Clustering-Based**: Algorithms like k-means or DBSCAN can be used, where data points that fall outside of clusters may be considered outliers.
  - **Nearest-Neighbor**: Methods like k-nearest neighbor (k-NN) can be used to detect anomalies by looking at the distance or similarity of an instance to its neighbors.

- **Isolation Forest**:
  - An algorithm that isolates anomalies instead of profiling normal data points. It works by randomly selecting features and then randomly selecting a split value between the maximum and minimum values of the selected feature.

- **Local Outlier Factor (LOF)**:

- This algorithm measures the local deviation of density of a given data point with respect to its neighbors. It considers as outliers the samples that have a substantially lower density than their neighbors.



Challenges in Anomaly Detection:

- **Defining Normal**: In many applications, it's hard to define what normal behavior is, and it may change over time.
- **High-Dimensional Data**: In high-dimensional spaces, distinguishing outliers from normal points becomes non-trivial due to the curse of dimensionality.
- **Evaluating Performance**: Without labeled data, it can be difficult to evaluate the performance of an anomaly detection system.

Thanks you for reading Clustering See you again .....